



Islamic University of Technology

Department of Computer Science and Engineering

Orthogonal Defect Classification on Defects4J

Using an LLM-Driven Scientific Approach

Thesis Defence · B.Sc. in Computer Science and Engineering

Presented by

Md. Sakib Hossain (210042133)

Mohammad Nahiyen Kabir (210042168)

Hasin Mahtab Alvee (210042174)

Supervisor

Lutfun Nahar Lota, Assistant Professor

Co-supervisor

Ishmam Tashdeed, Lecturer



Motivation: Classifying Defects in Defects4J

836

reproducible real bugs

17

Java projects [5]

- A standard benchmark for debugging research [6, 7]
- Defect types need different debugging strategies [1]
- Manual triage is accurate, but does not scale [3]
- LLMs can read all this evidence — **but should they answer at once, or investigate?**

WHAT EVERY DEFECTS4J BUG SHIPS WITH

Bug report

Failing tests + stack traces

Buggy source

Fixed source

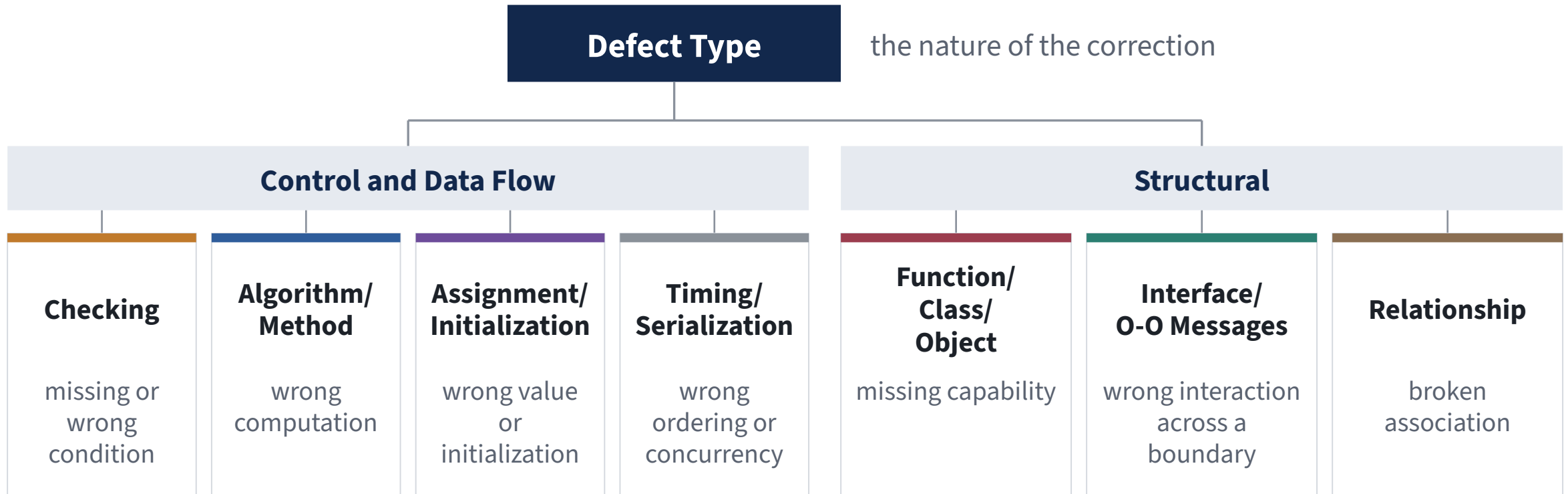
Fix diff

Every bug ships with its fix, so one bug can be classified **before** and **after** it.



Orthogonal Defect Classification (ODC)

ODC [4, 15] describes a defect when it is found (*opener*) and when it is fixed (*closer*). We classify one closer attribute:

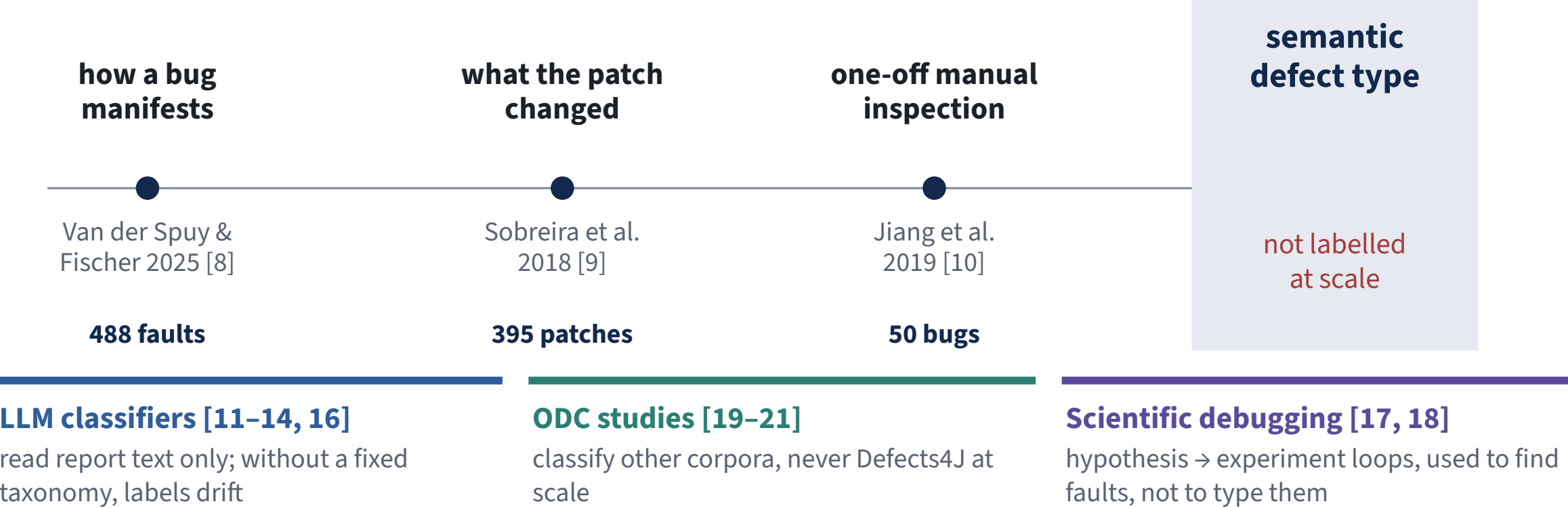


+ **Other** — escape category added in this study, allowed only with a written justification



Related Work and the Research Gap

WHAT PRIOR WORK ON DEFECTS4J ACTUALLY LABELS



Gap: no semantic defect types for Defects4J, no way to evaluate them, and no test of whether investigating helps.



Problem Statement

Classify Defects4J bugs into ODC defect types **from pre-fix evidence alone** — and evaluate that classification **without ground-truth labels**.

A SOLUTION HAS TO BE ... → HOW WE MEET IT

Context-aware code, tests, traces, report	→	one case file per bug	Semantic mechanism, not syntax	→	ODC defect type
Taxonomy-constrained a fixed, comparable label space	→	7 ODC types	Open to its own limits can say a bug does not fit	→	“Other” + justification
Scalable no manual labelling	→	LLM strategies	Evaluatable brings its own reference	→	post-fix reference



Core Idea: One Bug, Two Classifications

DEFECTS4J CHART_9

```
org.jfree.data.time.TimeSeries  
  
    endIndex = endIndex - 1;  
- if (endIndex < 0) {  
+ if ((endIndex < 0) || (endIndex < startIndex)) {  
    emptyRange = true;  
}
```

The fix widens a guard to catch an empty range.

Pre-fix

report · tests · trace · code

Triage: no fix exists yet.

Post-fix

the same evidence + the fix diff

Informed by the correction.

DEFECTS4J HAS NO ODC LABELS, SO WE DEFINE

Post-fix label

= the reference answer

Pre-fix vs post-fix

= does the triage label hold?

Labels differ

= drift

Agreement is not correctness: we measure how closely a pre-fix label reproduces a better-informed reference.



Research Questions

RQ1

Distribution

Which ODC types dominate, and does the mix differ by project?

p. 16

RQ2

Coverage

Do the seven types cover every bug, or is “Other” needed?

p. 16

RQ3

Accuracy

How often does the pre-fix label match the post-fix reference?

pp. 17–18

RQ4a

Taxonomy

What does the ODC taxonomy add over the model’s own words?

p. 19

RQ4b

Scientific strategy

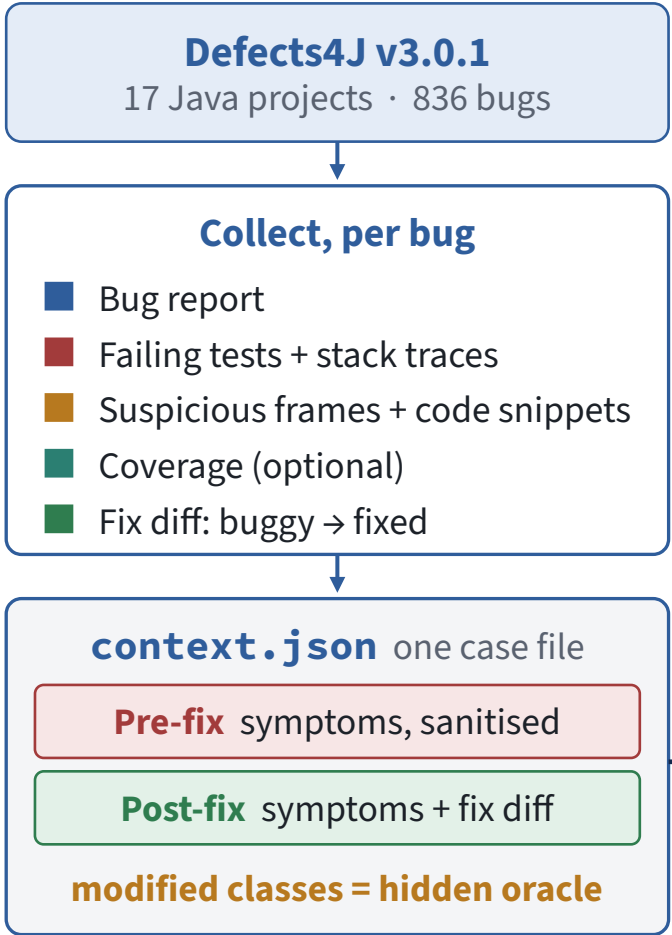
What does the scientific strategy add over a strong few-shot prompt?

pp. 20–22

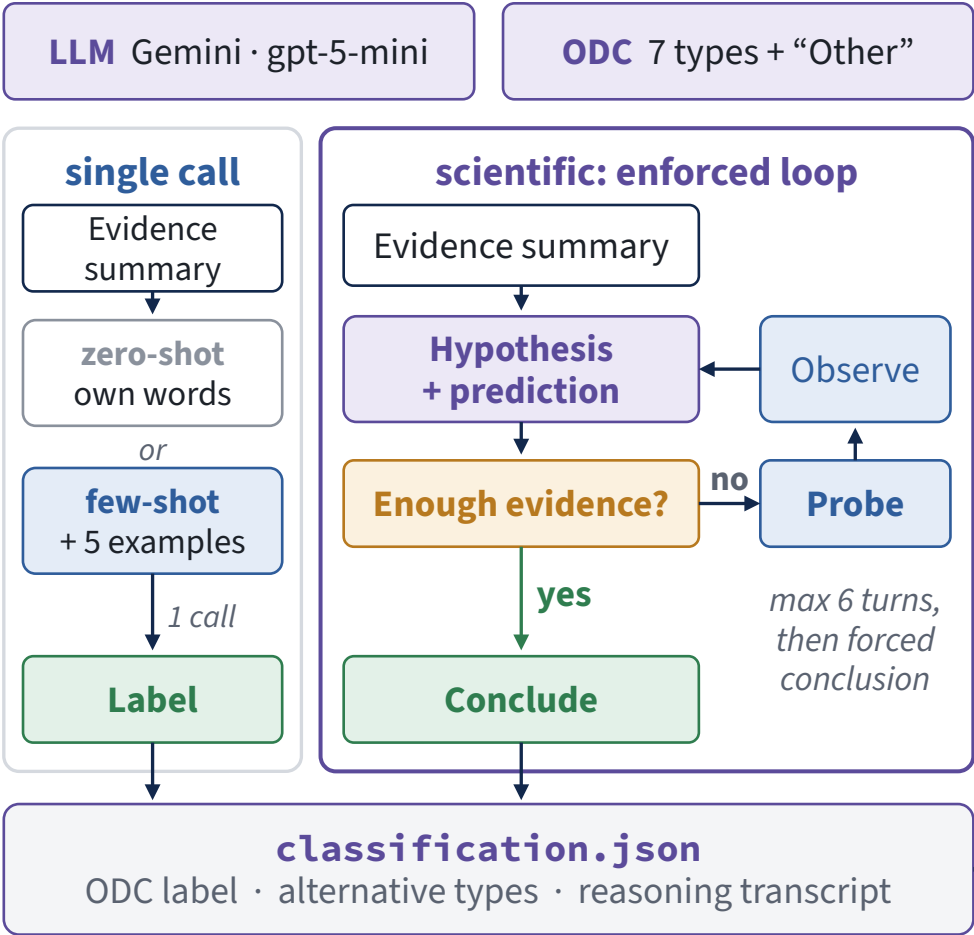


Pipeline Overview

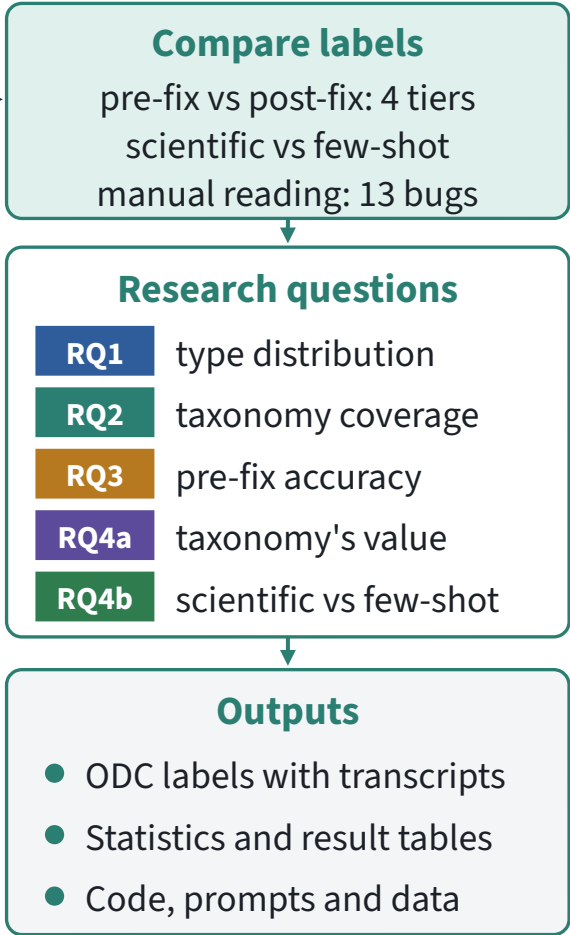
1 Preprocessing



2 Classification

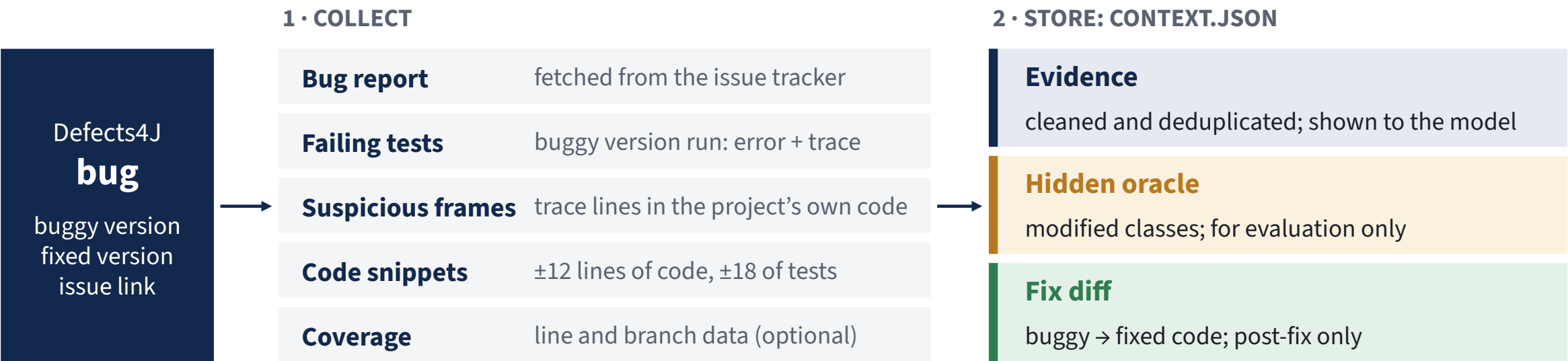


3 Evaluation





Evidence Collection: One Case File per Bug



3 • BUILD TWO CASE FILES • THE ONLY DIFFERENCE IS THE FIX DIFF



Every strategy and both LLMs read the same file. **few-shot** reads the evidence summary once; **scientific** reads the same summary, then probes for the held-back rest.



Classification Conditions and Models

A condition = **taxonomy** (which labels) × **strategy** (how it reasons), run by an **LLM**.

STRATEGY	TAXONOMY		
	free own words	closed 7 ODC types	open 7 types + “Other”
zero plain prompt	zero-free	—	—
few worked examples	—	few-closed	few-open
scientific hypothesis → probe	—	scientific-closed	scientific-open default

Five of nine combinations are valid: zero-shot uses no taxonomy, so it pairs only with free.

THIRD AXIS: THE LLM

Gemini 3.1 Flash-Lite
Google · main model
structured output enforced by the API

gpt-5-mini
OpenAI · second model
a reasoning model, used to cross-check

Only the model changes: same case files, prompts, taxonomy and validation.



Prompt Design: What Each Strategy Sends

Same in both: ODC taxonomy · anti-bias rules · the bug's evidence summary · JSON answer format

few-shot · one call, one answer

① **Instructions**

ODC definitions, a 7-question decision tree, 5 worked examples

② **The bug**

the evidence summary

*one call,
no extra evidence*

③ **Answer**

one JSON: type, alternatives, reasoning

scientific · up to 6 turns

① **Instructions**

ODC definitions, the list of 5 probes, turn rules

② **The bug**

the same summary; the rest is held back

EACH TURN

model hypothesis → prediction → probe request

pipeline returns what the probe found

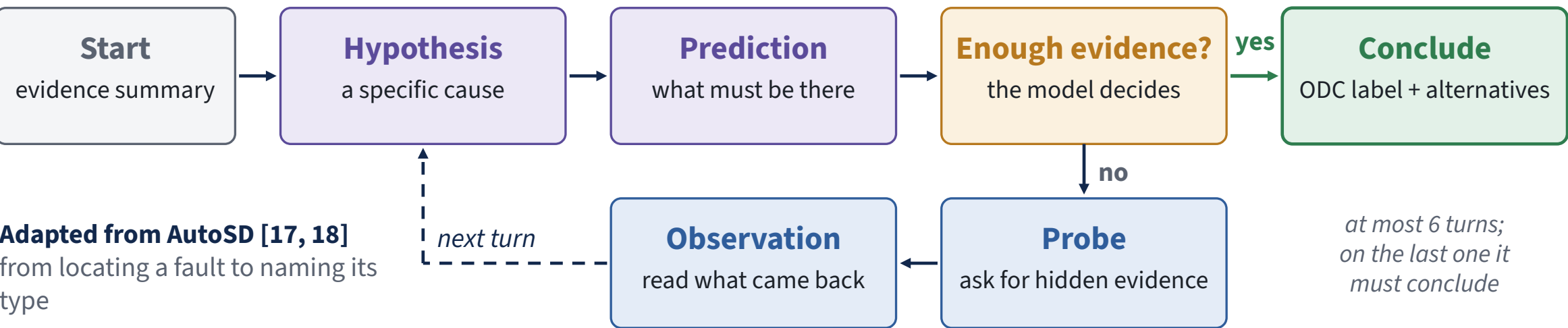
③ **Answer**

the same JSON + the full turn transcript



How the Scientific Strategy Reasons

THE SUMMARY IS READ ONCE · EVERY TURN THEN STARTS AT HYPOTHESIS



Adapted from AutoSD [17, 18]
from locating a fault to naming its type

FIVE PROBES: THE ONLY WAY TO SEE EVIDENCE HELD BACK FROM THE PROMPT

<code>list_evidence()</code> what evidence exists	<code>full_stack_trace(test)</code> full trace and message	<code>snippet(class)</code> source of a class	<code>coverage(class)</code> line and branch data	<code>bug_report()</code> the full report
--	---	--	--	--

The prediction is written before the evidence arrives, so a first guess cannot be rationalised afterwards.



Evaluation: How We Compare Two Labels

For each bug we compare its **pre-fix** label with its **post-fix** label, from strict to forgiving.

TIER 1

Strict match

Same ODC type?

pre

Checking

post

Checking

same type

>

TIER 2

Top-2 match

One label is the other's alternative?

pre

Checking

post

Algorithm

Algorithm was a listed alternative

>

TIER 3

Family match

Same ODC family?

pre

Checking

post

Assignment

both Control and Data Flow

TIER 4

Cohen's κ : is the agreement more than luck?

1

How we calculate it:

Over all bugs, we take the share whose pre-fix and post-fix labels match, then subtract the share that would match **by chance** given how often each run uses each type. κ = what is left $\div$ what could be left.

2

Why we need it:

Most bugs are Checking or Algorithm, so two runs agree often just by picking common types. κ removes that luck.

3

How the result slides show it:

In words, not numbers: “**agreement well beyond chance**”, “a real gain, not luck”. Exact values are in the book; scale from [27].

Illustration

chance

real

0%100%

observed agreement = chance + real

$\kappa = \text{real} \div \text{this span}$



Why the Post-Fix Label Is a Reliable Reference

The post-fix run is the same classifier given **one extra piece of evidence: the developer's fix**. Four reasons make it the better-informed rater.

1 It sees what defines the answer.

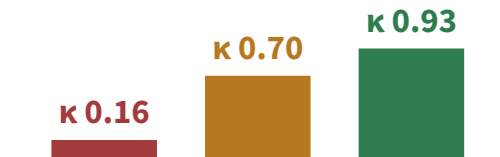
In ODC, the Defect Type is the nature of the correction [4, 15]. Pre-fix has to guess the correction from symptoms; post-fix can read it.

– buggy line

+ fixed line

2 More evidence makes raters agree.

Human raters agree far more when they see the code and the change than when they read only a fault description [22–26].



3 It points at one specific fault.

A buggy version holds about 9.2 faults [29]. The fix diff shows exactly the one the benchmark bug is about.



4 Nothing else changes.

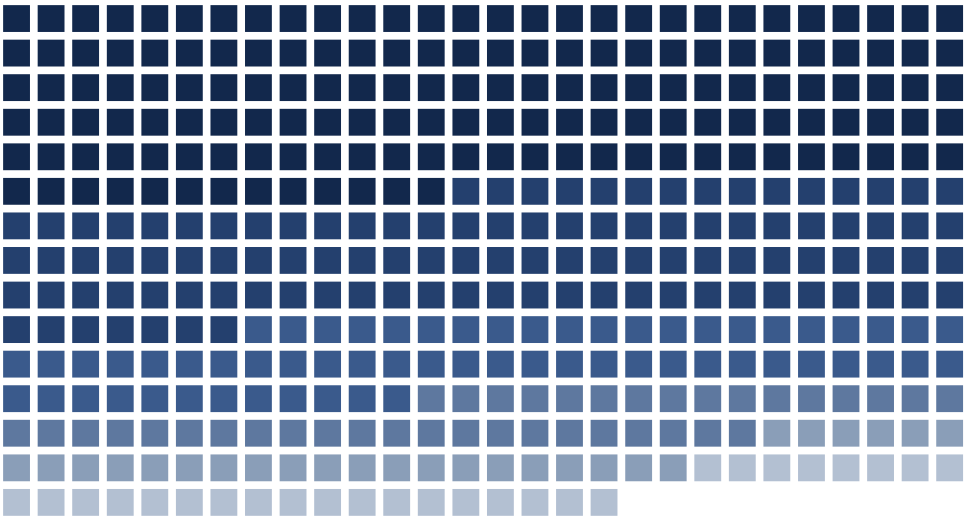
Same model, same prompt, same taxonomy, same evidence. Any difference between the two labels comes from the fix alone.





Experimental Setup

410 BUGS FROM 6 PROJECTS • ONE SQUARE PER BUG



- Closure 153
- Math 106
- Lang 61
- Mockito 38
- Chart 26
- Time 26

WHAT WE RAN

Main study • Gemini 3.1 Flash-Lite

410 bugs × scientific + few-shot × pre + post = 1,640 runs

Baseline for RQ4a • Gemini 3.1 Flash-Lite

410 bugs × zero-free × pre + post = 820 runs

Cross-model check • gpt-5-mini

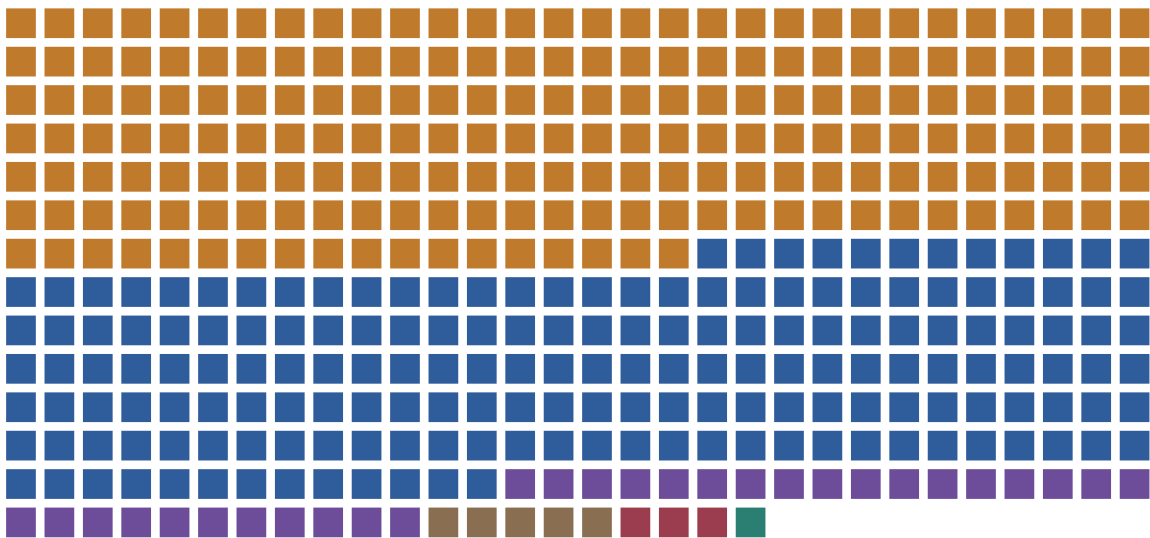
13 bugs × scientific + few-shot × pre + post = 52 runs

The 13 cross-model bugs are the ones we also read by hand. Reporting follows the LLM study guidelines [28].



RQ1–RQ2: Distribution and Coverage

RQ1 • 410 PRE-FIX LABELS, ONE SQUARE PER BUG



97.8% in three Control and Data Flow types; Timing never occurs. Scientific **keeps the same mix in every project**; few-shot’s mix shifts by project.

RQ2 • THE “OTHER” CATEGORY

0 of 1,640

classifications chose “Other”, although it was offered every time.

< 1 in 500

bugs could need “Other”, even in the worst case

6 of 7

ODC types actually used



RQ3: Agreement with the Post-Fix Reference

70.2% **288 of 410** pre-fix labels match the post-fix label exactly
scientific strategy, the pipeline default

ALL 410 BUGS • HOW CLOSE IS THE PRE-FIX LABEL?



near miss: the other label was listed as an alternative, or both share an ODC family

Both strategies

scientific288 of 410

few-shot273 of 410

Scientific also agrees more once lucky matches are discounted.

83.4% the realistic ceiling

Where it comes from: two scientific post-fix runs match on **342 of 410 bugs**.

What it means: the reference itself shifts on 1 bug in 6, so 100% is out of reach.

93.2% answer is in the output

For **382 of 410 bugs**, the post-fix type already appears in the pre-fix answer:

288as its main label

94as a listed alternative



RQ3: How Labels Change When the Fix Is Visible

PRE-FIX LABEL (ROW) → POST-FIX LABEL (COLUMN)

	Checking	Algorithm	Assignment	other types
Checking	148	37	8	5
Algorithm	38	119	10	8
Assignment	3	2	18	5
other types	3	2	1	3

outlined the label did not change **darker** more bugs

The two dark cells cancel out: 38 bugs move one way, 37 the other.

Algorithm/Method vs Checking is also the boundary human raters confuse [22]; the fix shows one fault of several [29].

29.8%

of bugs change label (122 of 410)

80.3%

of changes stay in these three types

About 5%

of the overall type mix shifts: the totals barely move

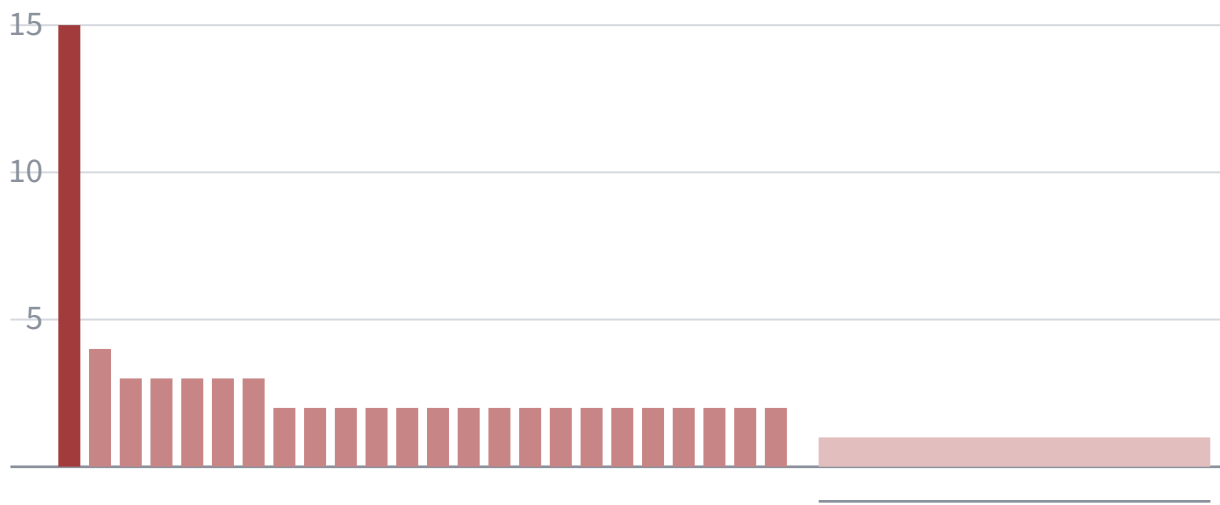


RQ4a: Contribution of the ODC Taxonomy

366 different labels

when the model writes its own (zero-free, 410 bugs)

“Null Pointer Dereference” 15 bugs



342 labels used exactly once

The same bug (Chart_9), written twice:
“Improper input validation logic”
“Incorrect boundary condition logic”

6 comparable labels

with the ODC taxonomy (scientific-open)



every bug lands in one of six buckets that can be counted

Condition	Same label, both modes
zero-free	15.1%
few-open	66.6%
scientific-open	70.2%

A fixed taxonomy lets us count anything at all.



RQ4b: Scientific Beats Few-Shot, Bug by Bug

73

bugs **only scientific** gets right

58

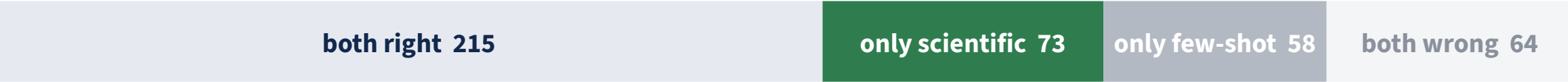
bugs **only few-shot** gets right

+15

more bugs right with scientific than with few-shot (73 – 58)

ALL 410 BUGS · DOES THE PRE-FIX LABEL MATCH THE POST-FIX LABEL?

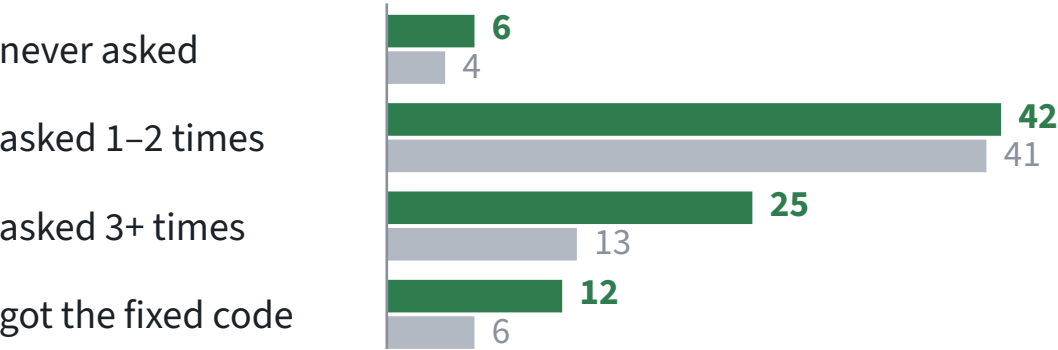
A real gain, not luck · statistically significant



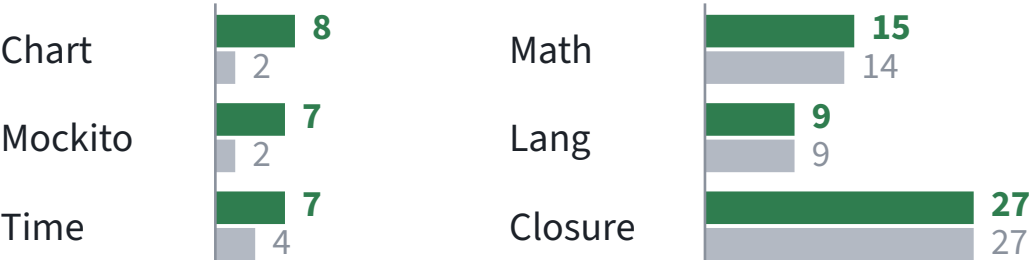
only scientific right

only few-shot right

MORE HIDDEN EVIDENCE ASKED FOR → MORE SCIENTIFIC WINS



SCIENTIFIC WINS AS OFTEN OR MORE, IN EVERY PROJECT





Qualitative Analysis of Thirteen Bugs

We read every artifact by hand: evidence, full reasoning transcript and developer’s fix, for both strategies and both models

1 How the classification goes wrong

	LLM said	correct
Symptom site Chart_17 followed where the error appeared	Checking	Algorithm
Report anchoring Math_90 copied the fix the report suggested	Checking	Interface
Patch shape Math_23 judged the patch by its looks	Assignment	Algorithm
Two valid fixes Lang_20 developer picked the other fix	Checking	Assignment

2 Where the developer had a choice

Math_90	could a Comparable check CHK chose a typed overload INT
Math_17	could a range check only CHK chose check + fallback CHK+ALG
Time_3	could better arithmetic ALG chose amount != 0 guards CHK
Lang_20	could a null check CHK chose a new capacity ASN

A fix repairs one of several faults [29] — for **77%** of changed labels, the other type is already a recorded alternative.



RQ4b: Why Scientific Wins — Evidence from the 13 Bugs

5 : 1

5 bugs vs 1 bug, all pre-fix

model	■ only scientific right	■ only few-shot right
Gemini	Math_17 Mockito_26 Chart_7	Math_104
gpt-5-mini	Time_3 Chart_11	none

WHAT THE TRANSCRIPTS SHOW

Time_3 gpt-5-mini scientific Checking few-shot Algorithm kept testing “a zero guard is missing” for six turns	Chart_11 gpt-5-mini scientific Assignment few-shot Algorithm narrowed its prediction to one wrong assignment	Math_17 Gemini scientific Checking few-shot Function had to name a concrete cause: a range check	Math_104 Gemini scientific Algorithm few-shot Assignment named the wrong constant, but chose Algorithm
--	---	---	---

green = the label that matches our manual reading

Same with both models → the pipeline Scientific beats few-shot pre-fix: 6 vs 4 of 13 Misleading clues can stop it too early (Chart_17)	Differs between models → the model gpt-5-mini asks for evidence about twice as often as Gemini Gemini trusts its first summary and stops early
--	---



Threats to Validity

Internal

The model can give a different answer each time it runs.
We removed fix hints from the pre-fix evidence, so it cannot see the answer.

External

We studied 6 of the 17 projects, all in Java, with one model on all 410 bugs.
Other projects or models may give different numbers; a second model ran on 13 bugs.

Conclusion

Answers vary between runs: two scientific runs agree on 342 of 410 post-fix labels.
One person did the manual reading of the 13 bugs.

Other

These bugs are public, so the model may have seen them during training.
The confidence score the model reports does not warn us when it is wrong.



Conclusion and Future Work

97.8%

three types (RQ1)

0 of 1,640

“Other” (RQ2)

70.2%

pre-fix match (RQ3)

73 vs 58

scientific vs few-shot wins
(RQ4)

CONTRIBUTIONS

- A taxonomy × strategy condition space
- Enforced scientific reasoning + transcripts
- Where scientific beats few-shot, bug by bug
- Evaluation without ground truth

FUTURE WORK

- Strengthen scientific's weak spots
- Pick scientific or few-shot per bug
- Human reference labels for a sample
- More models, Impact, Trigger, Age



References (1/3)

- [1] G. Catolino et al., “Not All Bugs Are the Same: Understanding, Characterizing, and Classifying Bug Types,” *Journal of Systems and Software*, vol. 152, 2019.
- [2] T. Hirsch and B. Hofer, “Using Textual Bug Reports to Predict the Fault Category of Software Bugs,” *Array*, vol. 15, 2022.
- [3] R. Andrade et al., “An Empirical Study on the Classification of Bug Reports with Machine Learning,” *arXiv preprint*, 2025.
- [4] R. Chillarege et al., “Orthogonal Defect Classification – A Concept for In-Process Measurements,” *IEEE Transactions on Software Engineering*, vol. 18, no. 11, 1992.
- [5] R. Just, D. Jalali, and M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” *Proc. International Symposium on Software Testing and Analysis (ISSTA)*, 2014.
- [6] M. N. Rafi et al., “Revisiting Defects4J for Fault Localization in Diverse Development Scenarios,” *IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025.
- [7] M. Martinez et al., “Automatic Repair of Real Bugs in Java: A Large-Scale Experiment on the Defects4J Dataset,” *Empirical Software Engineering*, vol. 22, no. 4, 2017.
- [8] A. Van der Spuy and B. Fischer, “An Anatomy of 488 Faults from Defects4J Based on the Control- and Data-Flow Graph Representations of Programs,” *Proc. 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2025.
- [9] V. Sobreira et al., “Dissection of a Bug Dataset: Anatomy of 395 Patches from Defects4J,” *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018.
- [10] J. Jiang, Y. Xiong, and X. Xia, “A Manual Inspection of Defects4J Bugs and Its Implications for Automatic Program Repair,” *Science China Information Sciences*, vol. 62, no. 10, 2019.



References (2/3)

- [11] G. Colavito et al., “Large Language Models for Issue Report Classification,” Ital-IA 2024 Thematic Workshops, CEUR Workshop Proceedings, vol. 3762, 2024.
- [12] A. Koyuncu, “Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study,” ACM Transactions on Software Engineering and Methodology, vol. 35, no. 3, 2026.
- [13] Y. Nong et al., “Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities,” arXiv preprint, 2024.
- [14] C. Li et al., “KnowBug: Enhancing Large Language Models with Bug Report Knowledge for Deep Learning Framework Bug Prediction,” Knowledge-Based Systems, vol. 305, 2024.
- [15] IBM, “Orthogonal Defect Classification v5.2 for Software Design and Code,” IBM Research, 2013.
- [16] X. Du et al., “LLM-BRC: A Large Language Model-Based Bug Report Classification Framework,” Software Quality Journal, vol. 32, no. 3, 2024.
- [17] S. Kang et al., “Explainable Automated Debugging via Large Language Model-driven Scientific Debugging,” Empirical Software Engineering, vol. 30, no. 2, 2025.
- [18] S. Kang, G. An, and S. Yoo, “A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization,” Proc. ACM on Software Engineering, vol. 1, no. FSE, 2024.
- [19] L. Huang et al., “AutoODC: Automated Generation of Orthogonal Defect Classifications,” Automated Software Engineering, vol. 22, no. 1, 2015.
- [20] F. Thung, D. Lo, and L. Jiang, “Automatic Defect Categorization,” Proc. 19th Working Conference on Reverse Engineering (WCRE), 2012.



References (3/3)

- [21] A. M. Esfahani, N. Kahani, and S. A. Ajila, “Understanding Defects in Generated Codes by Language Models,” 34th International Conference on Collaborative Advances in Software and Computing (CASCON), 2024.
- [22] K. Henningsson and C. Wohlin, “Assuring Fault Classification Agreement – An Empirical Evaluation,” Proc. 2004 International Symposium on Empirical Software Engineering (ISESE), 2004.
- [23] J. Hernández-González et al., “Learning to Classify Software Defects from Crowds: A Novel Approach,” Applied Soft Computing, vol. 62, 2018.
- [24] K. El Emam and I. Wiecek, “The Repeatability of Code Defect Classifications,” Proc. Ninth International Symposium on Software Reliability Engineering (ISSRE), 1998.
- [25] A. Rahman and E. Farhana, “An Exploratory Characterization of Bugs in COVID-19 Software Projects,” arXiv preprint, 2020.
- [26] J. Agnelo, N. Laranjeiro, and J. Bernardino, “Using Orthogonal Defect Classification to Characterize NoSQL Database Defects,” Journal of Systems and Software, vol. 159, 2020.
- [27] J. R. Landis and G. G. Koch, “The Measurement of Observer Agreement for Categorical Data,” Biometrics, vol. 33, no. 1, 1977.
- [28] S. Baltes et al., “Guidelines for Empirical Studies in Software Engineering involving Large Language Models,” Empirical Software Engineering, 2026.
- [29] D. Callaghan and B. Fischer, “Mining Bug Repositories for Multi-Fault Programs,” IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Data and Tool Showcase Track, 2025.



Thank you

Questions and discussion

Orthogonal Defect Classification on Defects4J Using an LLM-Driven Scientific Approach

Md. Sakib Hossain · Mohammad Nahiyen Kabir · Hasin Mahtab Alvee